GitHub Vision Navigator — Observations

Approach

The core idea is to make understanding the page the model's job and keep the automation layer simple. The browser layer exposes only what a human can see: a view of the screen and a mouse/keyboard. Every decision is made by showing Claude a screenshot and asking for the next action as strict JSON. The loop: screenshot at a fixed 1280×900 viewport (so pixel coordinates map deterministically) → send screenshot, goal, and recent history to the model → the model returns one action (click, type, press, scroll, extract, done, or fail) → execute and repeat until completion or a step cap.

As per the scope of this problem, which is to navigate GitHub and extract release information, the system prompt carries a hard-coded "release skill": an explicit, staged script to access the releases page of a repo then extract version, tag, and author in an exact format with precise field definitions.

The code is four modules: browser (eyes/hands), vision (model client), agent (loop), and navigate (CLI), so each is independently testable, and the vision provider is swappable by re-implementing one function.

What worked

- The agent code runs stably without crashing.
- The agent successfully extracts the correct data as per the requirements.
- The agent can work for any GitHub repository via a command-line argument.
- The agent accepts natural language prompts for flexible queries (e.g., "Find the latest React release and list its key features") and can extract some additional metadata.
- The selector-free contract is clean. The only interface between "what we see" and "what we do" is a pixel coordinate, so nothing in the code breaks when GitHub renames a class or restructures markup.
- The pure-logic core is solid and tested. The lenient JSON parser, the output normalizer, the deep-merge, and the completion heuristic are covered by 20 unit tests that pass with no network or API key.
- Graceful degradation. Vision API errors, unparseable responses, and failed actions (e.g. an out-of-viewport coordinate) are caught: the loop logs them and lets the model retry from a fresh screenshot rather than crashing.
- There are multiple flags for the user to choose from, depending on needs.

What didn't work (and potential fixes)

- Due to the undeterministic nature of LLMs, the agent occasionally gets stuck in a loop of deciding to execute the same action, though that action has already been executed the step prior. A loop detector that prompts the LLM to choose a different action if the action chosen is the same as the prior step could fix the issue, with the caveat that because this is a vision-based model, the pixels the LLM returns are often times near each other, but not identical. Engineering decisions must be made regarding the details of this loop detector.

Trade-offs

- To allow for adaptability to GitHub website changes, every action is decided via an LLM call, even the actions that are identical, no matter the user prompt (given the scope of the problem). The tradeoff is speed and cost for resistance against changes.

Limitations

- No authentication (public pages only, per the brief). Single-tab, synchronous Playwright — not built for concurrency.
- Coordinate precision. Vision models can be a few pixels off on small targets; clicking element centers and a generous viewport help, but dense UIs remain the main failure mode.
- The agent is only reliable within the scope of the problem, not a general-purpose web agent. The system prompt is hard-coded to guide the agent through an information retrieval process of the latest releases.
- The agent is undeterministic by nature, and thus, a run does not always succeed.

Future improvements

- Develop the “access the releases of a given repo” into a skill. Add an additional step before the classical agentic loop to determine the goal of the prompt and see if it matches an existing skill, or could use an existing skill. This can be expanded and generalized across the entire GitHub website, and potentially to other sites.
- Loop detection: if consecutive actions are near-identical, inject a corrective note so the model tries something different instead of grinding to the step cap.
- A self-check pass and an eval harness: verify extracted JSON against a fresh screenshot before “done,” and run the loop across several repos to report a measured success rate.